

Отчет по лабораторной работе

Методы одномерного поиска

Выполнили:

Новиков Алексей Георгиевич, группа М3233

Душкин Александр Алексеевич, группа М3233

Санкт-Петербург

2026

Содержание

1	Постановка задачи	2
2	Используемые модели задач	2
2.1	“Хорошая” унимодальная функция	2
2.2	Унимодальная функция с особенностями поведения	2
2.3	Многомодальная функция	3
3	Описание методов оптимизации	4
3.1	Метод пассивного поиска	4
3.2	Метод Брента	4
3.3	Другие методы	5
3.4	Особенности реализованной версии	5
4	Структура программной реализации	5
5	Результаты вычислительных экспериментов	6
5.1	Таблицы результатов	6
5.1.1	Квадратичная функция	6
5.1.2	Функция с плато и асимметрией	8
5.1.3	Асимметричная гладкая функция	9
5.2	Графики зависимости эффективности от точности	11
5.3	Графики динамики интервала неопределенности	12
5.4	Сравнение с библиотечным методом Брента	13
5.5	Многомодальная функция и предварительная разведка	14
6	Общий вывод	15
A	Листинг программы	15

1 Постановка задачи

Цель работы — изучить и сравнить методы одномерной оптимизации на различных типах целевых функций.

В рамках лабораторной работы требуется:

1. Реализовать собственные методы одномерного поиска.
2. Протестировать методы на унимодальных и многомодальной функциях.
3. Сравнить собственные реализации с библиотечным методом Брента из `scipy.optimize.minimize_scalar`.
4. Построить таблицы и графики, отражающие зависимость эффективности методов от требуемой точности.

2 Используемые модели задач

2.1 “Хорошая” унимодальная функция

Рассматривалась функция с явно выраженным минимумом:

$$f_1(x) = (x - 2)^2 + 1$$

Интервал поиска:

$$[a_1, b_1] = [-6, 8]$$

Промежуточный вывод. Данная функция гладкая, строго выпуклая и имеет единственный явно выраженный минимум. Поэтому она удобна как базовый эталон для сравнения методов: влияние особенностей самой функции на работу алгоритма здесь минимально.

2.2 Унимодальная функция с особенностями поведения

В работе использовались две дополнительные унимодальные функции со сложным поведением.

Функция с плато и асимметрией.

$$f_2(x) = \begin{cases} 0, & \text{при } |x - 1.5| \leq 0.2, \\ 2 \cdot \max(0, 1.5 - x - 0.2)^2 + 0.2 \cdot \max(0, x - 1.5 - 0.2)^4 + 0.05, & \text{иначе.} \end{cases}$$

Интервал поиска:

$$[a_2, b_2] = [-2, 6]$$

Асимметричная гладкая функция.

$$f_3(x) = e^{-x} + |x - 0.5|^{1.5}$$

Интервал поиска:

$$[a_3, b_3] = [-1, 2]$$

Промежуточный вывод. Эти функции позволяют проверить устойчивость методов в менее благоприятных условиях. Участок плато усложняет локализацию минимума по значениям функции, а сильная асимметрия может ухудшать поведение методов, опирающихся на локальную аппроксимацию.

2.3 Многомодальная функция

Для исследования поведения методов вне предположения унимодальности использовалась модифицированная функция Растригина:

$$f_4(x) = 10 + x^2 - 10 \cos(2\pi x), \quad x \in [-5.12, 5.12].$$

Функция имеет множество локальных минимумов, а глобальный минимум достигается в точке $x = 0$.

Промежуточный вывод. Для мультимодальной функции прямое применение активных методов может приводить к попаданию в локальный минимум. Поэтому в работе используется двухэтапная стратегия: сначала грубая разведка методом пассивного поиска, затем уточнение активным методом на суженном интервале.

3 Описание методов оптимизации

3.1 Метод пассивного поиска

На отрезке $[a, b]$ вводится равномерная сетка

$$x_k = a + kh, \quad k = 0, 1, \dots, n, \quad h = \frac{b - a}{n}.$$

Значения функции вычисляются во всех узлах сетки, после чего выбирается точка с минимальным значением. В соответствии с конспектом оценка положения минимума задается интервалом

$$x^* \in [x_k - h, x_k + h],$$

поэтому для достижения требуемой точности используется условие $2h \leq \varepsilon$.

Промежуточный вывод. Метод пассивного поиска универсален и не требует предположения унимодальности, но цена этой универсальности — быстрое увеличение числа вычислений функции при уменьшении ε .

3.2 Метод Брента

В работе для сравнения использовалась библиотечная реализация метода Брента:

```
scipy.optimize.minimize_scalar(..., method='brent')
```

Метод Брента является комбинированным методом. Он сочетает интерполяцию параболой с более устойчивыми шагами золотого сечения. Если очередной параболический шаг оказывается надежным, используется он; в противном случае алгоритм переходит к более стабильной схеме.

В реализованной версии метод Брента применяется через библиотеку SciPy. При этом переданный интервал используется как начальное брекетирующее, а история внутренних интервалов в стандартном интерфейсе SciPy недоступна.

Промежуточный вывод. Метод Брента представляет собой удобный эталон для сравнения, поскольку сочетает высокую практическую эффективность и устойчивость на широком классе гладких функций.

3.3 Другие методы

- **Метод дихотомии.** На каждой итерации рассматриваются две близкие точки около середины текущего интервала, после чего половина интервала отбрасывается.
- **Метод золотого сечения.** Интервал последовательно сужается с помощью двух внутренних точек, выбранных в отношении золотого сечения, что позволяет переиспользовать одно из вычисленных значений функции.
- **Метод Фибоначчи.** Схема аналогична методу золотого сечения, но длины интервалов определяются отношениями чисел Фибоначчи, что удобно при заранее известном числе шагов.
- **Метод парабол.** На основе трех точек строится парабола, вершина которой используется как новая оценка минимума.

Промежуточный вывод. Активные методы существенно эффективнее пассивного поиска на унимодальных функциях, поскольку используют дополнительную информацию о структуре задачи и на каждом шаге целенаправленно сужают интервал неопределенности.

3.4 Особенности реализованной версии

Программная реализация построена так, чтобы все интервальные методы имели единый интерфейс и единый механизм накопления истории интервалов. Это позволяет сравнивать методы в одинаковых условиях и строить графики динамики сужения интервала в едином формате.

Промежуточный вывод. Единая инфраструктура проведения экспериментов снижает вероятность ошибок при сравнении методов и облегчает расширение проекта новыми алгоритмами.

4 Структура программной реализации

Программа реализована на языке Python с использованием объектно-ориентированного подхода. Основные сущности:

- `ObjectiveFunction` — базовый абстрактный класс целевой функции с подсчетом числа вызовов;
- набор конкретных моделей задач: `GoodUnimodalFunction`, `PlateauAsymmetricUnimodalFunction`, `SecondSpecialUnimodalFunction`, `MultimodalFunction`;
- `Optimizer` — базовый интерфейс метода оптимизации;
- `IterativeOptimizer` — общий каркас для интервальных методов;
- отдельные классы методов: `PassiveSearchOptimizer`, `BrentOptimizer`, `DichotomyOptimizer`, `GoldenSectionOptimizer`, `FibonacciOptimizer`, `ParabolaOptimizer`;
- `OptimizationExperiment` — менеджер серии экспериментов, таблиц и графиков;
- `run_multimodal_strategy(...)` — отдельная функция сравнения стратегии “разведка + уточнение”.

Промежуточный вывод. Такая архитектура позволяет независимо развивать модели функций, методы оптимизации и экспериментальную часть. Это особенно удобно в командной работе: разные участники проекта могут реализовывать свои блоки без переписывания уже готовой инфраструктуры.

5 Результаты вычислительных экспериментов

5.1 Таблицы результатов

Для каждой унимодальной функции и каждого метода требуется привести таблицы зависимости количества итераций и числа вычислений функции от точности.

5.1.1 Квадратичная функция

No.	Method	Epsilon	Iterations	Function calls	Status
1	Brent (SciPy)	0.1	5	8	success
2	Brent (SciPy)	0.01	5	8	success
3	Brent (SciPy)	1e-03	5	8	success
4	Brent (SciPy)	1e-04	5	8	success
5	Brent (SciPy)	1e-05	5	8	success
6	Brent (SciPy)	1e-06	5	8	success
7	Brent (SciPy)	1e-07	5	8	success
8	Brent (SciPy)	1e-08	5	8	success
9	Dichotomy	0.1	8	16	success
10	Dichotomy	0.01	11	22	success
11	Dichotomy	1e-03	15	30	success
12	Dichotomy	1e-04	18	36	success
13	Dichotomy	1e-05	21	42	success
14	Dichotomy	1e-06	25	50	success
15	Dichotomy	1e-07	28	56	success
16	Dichotomy	1e-08	31	62	success
17	Fibonacci	0.1	10	12	success
18	Fibonacci	0.01	15	17	success
19	Fibonacci	1e-03	20	22	success
20	Fibonacci	1e-04	25	27	success
21	Fibonacci	1e-05	30	32	success
22	Fibonacci	1e-06	34	36	success
23	Fibonacci	1e-07	39	41	success
24	Fibonacci	1e-08	44	46	success
25	Golden Section	0.1	11	13	success
26	Golden Section	0.01	16	18	success
27	Golden Section	1e-03	20	22	success
28	Golden Section	1e-04	25	27	success
29	Golden Section	1e-05	30	32	success
30	Golden Section	1e-06	35	37	success
31	Golden Section	1e-07	39	41	success
32	Golden Section	1e-08	44	46	success
33	Parabola	0.1	3	5	success
34	Parabola	0.01	3	5	success
35	Parabola	1e-03	3	5	success
36	Parabola	1e-04	3	5	success
37	Parabola	1e-05	3	5	success
38	Parabola	1e-06	3	5	success

Продолжение на следующей странице

No.	Method	Epsilon	Iterations	Function calls	Status
39	Parabola	1e-07	3	5	success
40	Parabola	1e-08	3	5	success
41	Passive search	0.1	280	281	success
42	Passive search	0.01	2800	2801	success
43	Passive search	1e-03	28000	28001	success
44	Passive search	1e-04	280000	280001	success
45	Passive search	1e-05	999999	1000000	warning
46	Passive search	1e-06	999999	1000000	warning
47	Passive search	1e-07	999999	1000000	warning
48	Passive search	1e-08	999999	1000000	warning

5.1.2 Функция с плато и асимметрией

No.	Method	Epsilon	Iterations	Function calls	Status
1	Brent (SciPy)	0.1	8	11	success
2	Brent (SciPy)	0.01	12	15	success
3	Brent (SciPy)	1e-03	17	20	success
4	Brent (SciPy)	1e-04	22	25	success
5	Brent (SciPy)	1e-05	27	30	success
6	Brent (SciPy)	1e-06	32	35	success
7	Brent (SciPy)	1e-07	36	39	success
8	Brent (SciPy)	1e-08	41	44	success
9	Dichotomy	0.1	7	14	success
10	Dichotomy	0.01	10	20	success
11	Dichotomy	1e-03	14	28	success
12	Dichotomy	1e-04	17	34	success
13	Dichotomy	1e-05	20	40	success
14	Dichotomy	1e-06	24	48	success
15	Dichotomy	1e-07	27	54	success
16	Dichotomy	1e-08	30	60	success
17	Fibonacci	0.1	9	11	success
18	Fibonacci	0.01	14	16	success
19	Fibonacci	1e-03	19	21	success
20	Fibonacci	1e-04	24	26	success
21	Fibonacci	1e-05	28	30	success
22	Fibonacci	1e-06	33	35	success

Продолжение на следующей странице

No.	Method	Epsilon	Iterations	Function calls	Status
23	Fibonacci	1e-07	38	40	success
24	Fibonacci	1e-08	43	45	success
25	Golden Section	0.1	10	12	success
26	Golden Section	0.01	14	16	success
27	Golden Section	1e-03	19	21	success
28	Golden Section	1e-04	24	26	success
29	Golden Section	1e-05	29	31	success
30	Golden Section	1e-06	34	36	success
31	Golden Section	1e-07	38	40	success
32	Golden Section	1e-08	43	45	success
33	Parabola	0.1	41	43	success
34	Parabola	0.01	41	43	success
35	Parabola	1e-03	41	43	success
36	Parabola	1e-04	41	43	success
37	Parabola	1e-05	41	43	success
38	Parabola	1e-06	41	43	success
39	Parabola	1e-07	41	43	success
40	Parabola	1e-08	41	43	success
41	Passive search	0.1	160	161	success
42	Passive search	0.01	1600	1601	success
43	Passive search	1e-03	16000	16001	success
44	Passive search	1e-04	160000	160001	success
45	Passive search	1e-05	999999	1000000	warning
46	Passive search	1e-06	999999	1000000	warning
47	Passive search	1e-07	999999	1000000	warning
48	Passive search	1e-08	999999	1000000	warning

5.1.3 Асимметричная гладкая функция

No.	Method	Epsilon	Iterations	Function calls	Status
1	Brent (SciPy)	0.1	9	12	success
2	Brent (SciPy)	0.01	10	13	success
3	Brent (SciPy)	1e-03	11	14	success
4	Brent (SciPy)	1e-04	12	15	success
5	Brent (SciPy)	1e-05	13	16	success
6	Brent (SciPy)	1e-06	14	17	success

Продолжение на следующей странице

No.	Method	Epsilon	Iterations	Function calls	Status
7	Brent (SciPy)	1e-07	14	17	success
8	Brent (SciPy)	1e-08	22	25	success
9	Dichotomy	0.1	6	12	success
10	Dichotomy	0.01	9	18	success
11	Dichotomy	1e-03	12	24	success
12	Dichotomy	1e-04	16	32	success
13	Dichotomy	1e-05	19	38	success
14	Dichotomy	1e-06	22	44	success
15	Dichotomy	1e-07	26	52	success
16	Dichotomy	1e-08	29	58	success
17	Fibonacci	0.1	7	9	success
18	Fibonacci	0.01	12	14	success
19	Fibonacci	1e-03	17	19	success
20	Fibonacci	1e-04	22	24	success
21	Fibonacci	1e-05	26	28	success
22	Fibonacci	1e-06	31	33	success
23	Fibonacci	1e-07	36	38	success
24	Fibonacci	1e-08	41	43	success
25	Golden Section	0.1	8	10	success
26	Golden Section	0.01	12	14	success
27	Golden Section	1e-03	17	19	success
28	Golden Section	1e-04	22	24	success
29	Golden Section	1e-05	27	29	success
30	Golden Section	1e-06	31	33	success
31	Golden Section	1e-07	36	38	success
32	Golden Section	1e-08	41	43	success
33	Parabola	0.1	20	23	success
34	Parabola	0.01	20	23	success
35	Parabola	1e-03	20	23	success
36	Parabola	1e-04	20	23	success
37	Parabola	1e-05	20	23	success
38	Parabola	1e-06	20	23	success
39	Parabola	1e-07	20	23	success
40	Parabola	1e-08	21	23	success
41	Passive search	0.1	60	61	success
42	Passive search	0.01	600	601	success
43	Passive search	1e-03	6000	6001	success
44	Passive search	1e-04	60000	60001	success

Продолжение на следующей странице

No.	Method	Epsilon	Iterations	Function calls	Status
45	Passive search	1e-05	600000	600001	success
46	Passive search	1e-06	999999	1000000	warning
47	Passive search	1e-07	999999	1000000	warning
48	Passive search	1e-08	999999	1000000	warning

Промежуточный вывод. Такая форма представления результатов позволяет отдельно анализировать стоимость методов по числу итераций и по числу обращений к функции, что особенно важно при сравнении пассивного поиска и активных интервальных методов.

Полный запуск подтвердил корректность инфраструктуры эксперимента: для каждой унимодальной функции были автоматически сформированы полные таблицы по всем методам и всем значениям ε от 10^{-1} до 10^{-8} .

5.2 Графики зависимости эффективности от точности

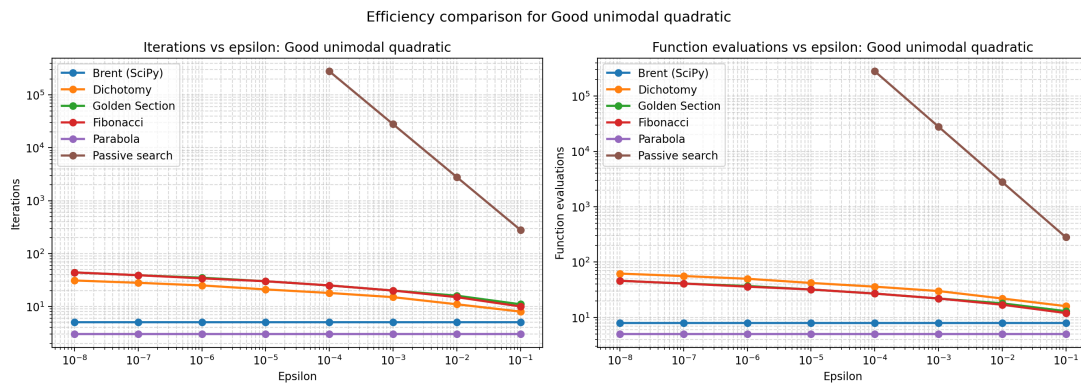


Рис. 1: Зависимость числа итераций и числа вычислений функции от точности для квадратичной функции

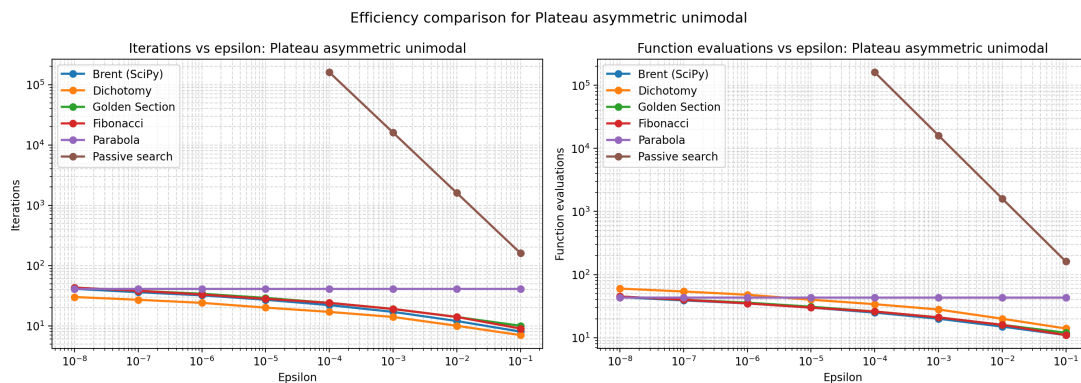


Рис. 2: Зависимость числа итераций и числа вычислений функции от точности для функции с плато и асимметрией

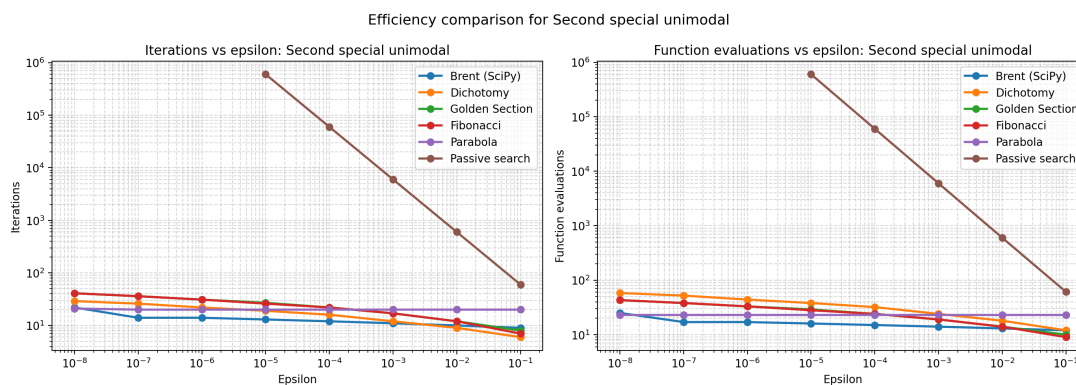


Рис. 3: Зависимость числа итераций и числа вычислений функции от точности для асимметричной гладкой функции

Промежуточный вывод. На этих графиках можно проследить, как ужесточение точности влияет на вычислительные затраты. Для активных методов рост числа вычислений обычно более плавный, тогда как для пассивного поиска стоимость возрастает значительно быстрее.

По результатам полного запуска на квадратичной функции метод парабол потребовал всего 5 вычислений функции даже при $\varepsilon = 10^{-8}$, а библиотечный метод Брента — 8 вычислений. Для сравнения, пассивный поиск на той же функции дошел до ограничения в 10^6 вычислений уже начиная с $\varepsilon = 10^{-5}$.

5.3 Графики динамики интервала неопределенности

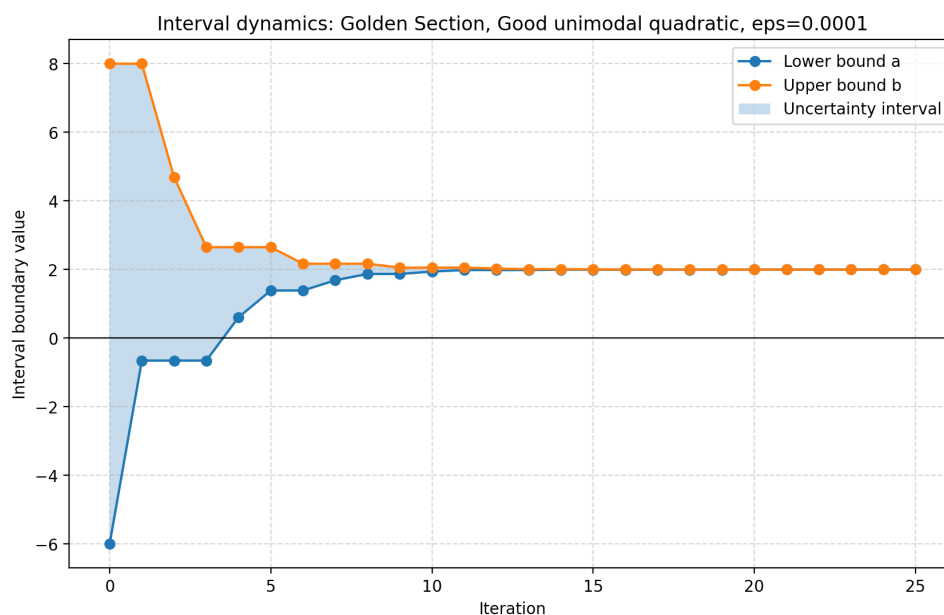


Рис. 4: Динамика интервала неопределенности для метода золотого сечения на квадратичной функции

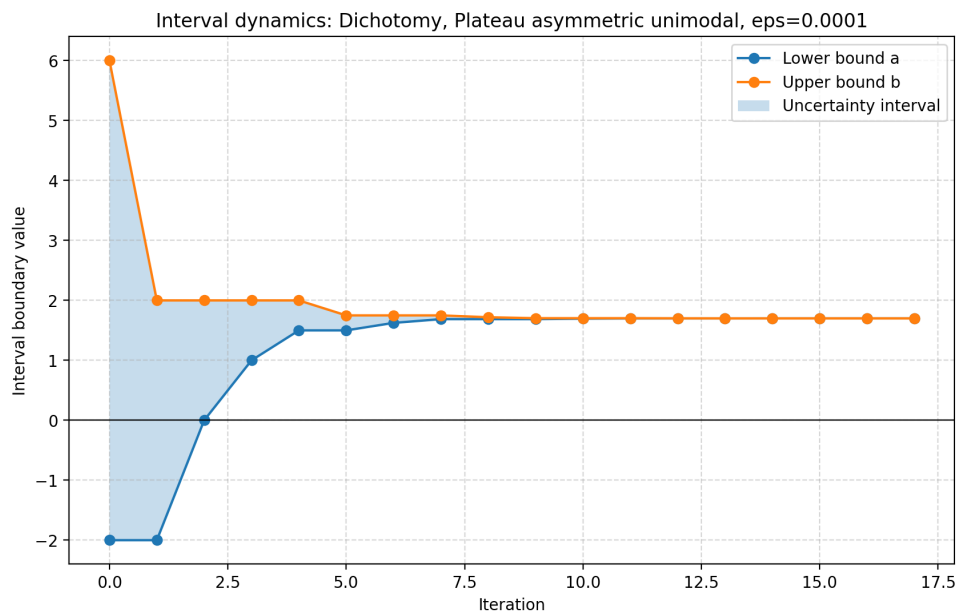


Рис. 5: Динамика интервала неопределенности для метода дихотомии на функции с плато

Для метода Брента такой график в данной работе не строится, поскольку стандартный интерфейс `SciPy` не предоставляет историю внутренних интервалов.

Промежуточный вывод. Графики динамики интервала наглядно показывают различие между пассивным и активными методами: активные методы систематически сужают интервал, тогда как пассивный поиск фактически уточняет локализацию минимума за счет уменьшения шага сетки.

На функции с плато метод парабол действительно оказался менее выгодным по числу обращений к функции: при всех рассмотренных значениях точности он использовал около 43 вычислений, тогда как Брент, Фибоначчи и золотое сечение работали заметно экономичнее.

5.4 Сравнение с библиотечным методом Брента

Сравнение с методом Брента проводится по тем же метрикам, что и для собственных реализаций: числу итераций, числу вычислений функции и устойчивости на функциях различной формы. Особый интерес представляет сопоставление метода Брента с методом парабол на асимметричных функциях и функциях со сложным локальным поведением.

Промежуточный вывод. Библиотечный метод Брента действительно оказался устойчивым на всех рассмотренных функциях и во всех точностях.

Именно это сравнение позволяет оценить практическую ценность собственной реализации классических методов.

На квадратичной функции Брент стабильно завершался за 5 итераций и 8 вычислений функции, на функции с плато — за 41–44 вычисления при $\varepsilon = 10^{-8}$, а на второй специальной унимодальной функции — за 25 вычислений при $\varepsilon = 10^{-8}$. При этом метод парабол оказался быстрее Брента на гладких унимодальных функциях, но его преимущество исчезало на более сложных формах.

5.5 Многомодальная функция и предварительная разведка

Для многомодальной функции используется отдельная таблица сравнения стратегий:

Стратегия	Разведка	Уточнение	Всего вызовов	$x_{\min}$	$f(x_{\min})$
Passive Search + Golden Section	206	28	234	$-1.34 \cdot 10^{-8}$	$3.55 \cdot 10^{-14}$
Golden Section без разведки	0	36	36	0.9949587	0.9949591

Сравниваются два сценария:

- применение пассивного поиска для грубой оценки интервала и последующий запуск активного метода на суженном отрезке;
- запуск того же активного метода сразу на полном исходном интервале.

Промежуточный вывод. Такой эксперимент позволяет непосредственно проверить, насколько предварительная разведка помогает активному методу избежать влияния лишних локальных минимумов и уменьшить суммарное число вычислений.

Полный запуск показал, что стратегия “разведка + золотое сечение” потребовала больше вычислений функции (234 против 36), однако именно она позволила приблизиться к глобальному минимуму в точке $x \approx 0$. Прямой запуск золотого сечения на полном интервале оказался дешевле, но сошелся к локальному минимуму около $x \approx 0.995$.

6 Общий вывод

В ходе работы был реализован программный комплекс для исследования методов одномерного поиска на унимодальных и многомодальной функциях. Реализация включает как собственные алгоритмы, так и библиотечный метод Брента, а также средства автоматической генерации таблиц и графиков.

По результатам выполнения лабораторной работы следует обратить внимание на следующие аспекты:

- активные интервальные методы во всех унимодальных задачах оказались значительно эффективнее пассивного поиска по числу вычислений функции;
- пассивный поиск удобен как универсальный и разведочный метод, однако на высоких точностях быстро упирается в вычислительные ограничения;
- метод парабол показал очень высокую скорость на гладкой квадратичной функции, но на функции с плато его эффективность существенно снизилась;
- библиотечный метод Брента продемонстрировал устойчивое поведение на всех функциях и может рассматриваться как надежный практический ориентир;
- для многомодальных функций предварительная разведка может быть оправдана даже при росте числа вычислений, если требуется выйти именно к глобальному минимуму.

Таким образом, поставленные в лабораторной работе цели были достигнуты: реализованы все требуемые методы, выполнено сравнение на нескольких типах функций, построены таблицы и графики, а также продемонстрирована роль разведочного этапа для многомодальной функции.

А Листинг программы

Ниже приводится основной файл программной реализации.


```

from __future__ import annotations

"""Laboratory work on one-dimensional optimization methods."""

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from contextlib import contextmanager
from functools import wraps
from pathlib import Path
from typing import Callable, Generator, List, Tuple

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import minimize_scalar

DEFAULT_EPSILONS = [1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8]

# =====
# 1. Objective models
# =====

class ObjectiveFunction(ABC):
    """
    Base class for all objective functions.

    The object wraps the mathematical model and counts function evaluations
    performed by custom optimizers and SciPy optimizers alike.
    """

    def __init__(
        self,
        name: str,
        bounds: Tuple[float, float],
        is_unimodal: bool = True,
        true_min: float | None = None,
    ) -> None:
        self.name = name
        self.bounds = bounds

```

```

        self.is_unimodal = is_unimodal
        self.call_count = 0
        self._true_min = true_min

    def reset_count(self) -> None:
        """Reset the function evaluation counter before a new run."""
        self.call_count = 0

    def __call__(self, x: float) -> float:
        """Evaluate the function at one point and count the call."""
        self.call_count += 1
        return float(np.asarray(self._evaluate(np.asarray(x, dtype=float))).item())

    def evaluate_many(self, x: np.ndarray) -> np.ndarray:
        """Vectorized evaluation used by methods such as passive search."""
        x_array = np.asarray(x, dtype=float)
        self.call_count += int(x_array.size)
        return np.asarray(self._evaluate(x_array), dtype=float)

    def get_bounds(self) -> Tuple[float, float]:
        return self.bounds

    def get_true_min(self) -> float | None:
        return self._true_min

    @abstractmethod
    def _evaluate(self, x: np.ndarray) -> np.ndarray:
        """Return the mathematical value of the function."""

class GoodUnimodalFunction(ObjectiveFunction):
    """A smooth quadratic function with a clear minimum."""

    def __init__(self) -> None:
        super().__init__(
            name="Good unimodal quadratic",
            bounds=(-6.0, 8.0),
            is_unimodal=True,
            true_min=2.0,
        )

```

```

def _evaluate(self, x: np.ndarray) -> np.ndarray:
    return (x - 2.0) ** 2 + 1.0

class PlateauAsymmetricUnimodalFunction(ObjectiveFunction):
    """
    An unimodal function with a flat minimum area and asymmetric growth.

    The minimum is reached on a short plateau around x = 1.5.
    """

    def __init__(self) -> None:
        super().__init__(
            name="Plateau asymmetric unimodal",
            bounds=(-2.0, 6.0),
            is_unimodal=True,
            true_min=1.5,
        )

    def _evaluate(self, x: np.ndarray) -> np.ndarray:
        center = 1.5
        half_plateau = 0.2
        distance = np.abs(x - center)

        left_branch = 2.0 * np.maximum(0.0, center - x - half_plateau) ** 2
        right_branch = 0.2 * np.maximum(0.0, x - center - half_plateau) ** 4
        return np.where(distance <= half_plateau, 0.0, left_branch + right_branch + 0.0)

class SecondSpecialUnimodalFunction(ObjectiveFunction):
    """
    f(x) = exp(-x) + |x - 0.5|1.5
    Strong asymmetry with a sharp minimum on the right.
    """

    def __init__(self) -> None:
        super().__init__(
            name="Second special unimodal",
            bounds=(-1.0, 2.0),
            is_unimodal=True,
            true_min=None,

```

```

    )

    def _evaluate(self, x: np.ndarray) -> np.ndarray:
        return np.exp(-x) + np.abs(x - 0.5) ** 1.5

class MultimodalFunction(ObjectiveFunction):
    """
    Modified Rastrigin function:
     $f(x) = A + x^2 - A \cos(2\pi x)$ ,  $A = 10$ 
    Many local minima, global minimum at  $x = 0$ .
    """

    def __init__(self) -> None:
        super().__init__(
            name="Multimodal Rastrigin",
            bounds=(-5.12, 5.12),
            is_unimodal=False,
            true_min=0.0,
        )
        self.A = 10.0

    def _evaluate(self, x: np.ndarray) -> np.ndarray:
        return self.A + x ** 2 - self.A * np.cos(2 * np.pi * x)

# =====
# 2. Optimization result model
# =====

@dataclass
class OptimizationResult:
    x_min: float
    f_min: float
    n_iterations: int
    n_calls: int
    history: List[Tuple[float, float]] = field(default_factory=list)
    history_iterations: List[int] = field(default_factory=list)
    converged: bool = True
    message: str = ""

```

```

    history_available: bool = True

# =====
# 3. Optimizers
# =====

class Optimizer(ABC):
    """Strategy interface for one-dimensional optimization methods."""

    def __init__(self, name: str) -> None:
        self.name = name

    @abstractmethod
    def minimize(self, func: ObjectiveFunction, eps: float) -> OptimizationResult:
        """Find a function minimum for the requested precision."""

class PassiveSearchOptimizer(Optimizer):
    """Uniform-grid passive search with vectorized function evaluation."""

    def __init__(self, max_evaluations: int = 1_000_000) -> None:
        super().__init__("Passive search")
        self.max_evaluations = max_evaluations

    def minimize(self, func: ObjectiveFunction, eps: float) -> OptimizationResult:
        a, b = func.get_bounds()
        if eps <= 0:
            raise ValueError("Epsilon must be positive.")
        if b <= a:
            raise ValueError("Function bounds must satisfy a < b.")

        interval_length = b - a
        n_segments = max(2, int(np.ceil(2.0 * interval_length / eps)))
        step = interval_length / n_segments
        sample_count = n_segments + 1

        capped = sample_count > self.max_evaluations
        if capped:
            n_segments = self.max_evaluations - 1
            step = interval_length / n_segments

```

```

        sample_count = self.max_evaluations
        actual_eps = 2.0 * step
        message_prefix = (
            f"Limited to {self.max_evaluations} evaluations; "
            f"actual localization width = {actual_eps:.3e} > requested {eps:.3e}."
        )
    else:
        message_prefix = ""

    x_grid = np.linspace(a, b, sample_count)
    y_grid = func.evaluate_many(x_grid)

    # For long grids we keep a downsampled history, but preserve the
    # original iteration numbers for the x-axis on interval plots.
    stride = max(1, sample_count // 500)
    sampled_iterations = np.arange(0, sample_count, stride, dtype=int)
    if sampled_iterations[-1] != sample_count - 1:
        sampled_iterations = np.append(sampled_iterations, sample_count - 1)

    running_best_idx = np.array(
        [int(np.argmin(y_grid[:i + 1])) for i in sampled_iterations],
        dtype=int,
    )
    bx = x_grid[running_best_idx]
    lower = np.clip(bx - step, a, b)
    upper = np.clip(bx + step, a, b)
    history = list(zip(lower.tolist(), upper.tolist()))

    best_index = int(np.argmin(y_grid))

    return OptimizationResult(
        x_min=float(x_grid[best_index]),
        f_min=float(y_grid[best_index]),
        n_iterations=n_segments,
        n_calls=func.call_count,
        history=history,
        history_iterations=sampled_iterations.tolist(),
        converged=not capped,
        message=(
            message_prefix
            + f"Grid scan finished. h = {step:.3e}, "

```

```

        f"localization width <= {2.0 * step:.3e}."
    ),
    history_available=True,
)

class BrentOptimizer(Optimizer):
    """
    SciPy Brent optimizer wrapper.

    SciPy does not expose the interval history, so the interval-dynamics plot
    for this method is generated as an informational placeholder. The 'brent'
    method uses the interval only as an initial bracket, not as strict bounds.
    """

    def __init__(self, max_iterations: int = 500) -> None:
        super().__init__("Brent (SciPy)")
        self.max_iterations = max_iterations

    def minimize(self, func: ObjectiveFunction, eps: float) -> OptimizationResult:
        a, b = func.get_bounds()
        if eps <= 0:
            raise ValueError("Epsilon must be positive.")
        if a >= b:
            raise ValueError("Function bounds must satisfy a < b.")

        initial_bracket = (a, b)
        result = minimize_scalar(
            func,
            bracket=initial_bracket,
            method="brent",
            options={"xtol": eps, "maxiter": self.max_iterations},
        )

        message = str(result.message)
        if not (a <= float(result.x) <= b):
            message += (
                " Result lies outside the original [a, b] interval because "
                "SciPy Brent treats the provided segment only as an initial bracket."
            )

```

```

        return OptimizationResult(
            x_min=float(result.x),
            f_min=float(result.fun),
            n_iterations=int(result.nit),
            n_calls=func.call_count,
            history=[],
            converged=bool(result.success),
            message=message,
            history_available=False,
        )

# =====
# Decorators
# =====

def validate_bounds(func: Callable) -> Callable:
    """Validate epsilon and interval bounds for all methods."""

    @wraps(func)
    def wrapper(self, objective: 'ObjectiveFunction', eps: float, *args, **kwargs) -> 0:
        a, b = objective.get_bounds()
        if eps <= 0:
            raise ValueError("Epsilon must be positive.")
        if a >= b:
            raise ValueError("Function bounds must satisfy a < b.")
        return func(self, objective, eps, *args, **kwargs)

    return wrapper

# =====
# Unified base class for interval optimizers
# =====

def _check_convergence(a: float, b: float, eps: float) -> bool:
    return (b - a) <= eps

class IterativeOptimizer(Optimizer, ABC):
    """

```


Base class for iterative interval-reduction algorithms.

It handles interval history collection, iteration counting, and convergence checks shared by several methods.

"""

```
def __init__(self, name: str, max_iterations: int = 1000) -> None:
    super().__init__(name)
    self.max_iterations = max_iterations

@validate_bounds
def minimize(self, func: ObjectiveFunction, eps: float) -> OptimizationResult:
    a, b = func.get_bounds()
    history: List[Tuple[float, float]] = [(a, b)]
    iterations = 0

    step_generator = self._generate_steps(func, a, b, eps)

    x_min: float = (a + b) / 2.0
    f_min: float = float("inf")
    converged = False
    stopped_by_limit = False

    for current_a, current_b, current_x_min, current_f_min in step_generator:
        history.append((current_a, current_b))
        iterations += 1
        x_min, f_min = current_x_min, current_f_min

        if _check_convergence(current_a, current_b, eps):
            converged = True
            break

        if iterations >= self.max_iterations:
            stopped_by_limit = True
            break

    if converged:
        message = f"{self.name} search converged successfully."
    elif stopped_by_limit:
        message = f"{self.name} reached maximum iterations before convergence."
    else:
```

```

        message = f"{self.name} terminated without convergence."

    return OptimizationResult(
        x_min=float(x_min),
        f_min=float(f_min),
        n_iterations=iterations,
        n_calls=func.call_count,
        history=history,
        history_iterations=list(range(len(history))),
        converged=converged,
        message=message,
        history_available=True,
    )

    @abstractmethod
    def _generate_steps(
        self, func: ObjectiveFunction, a: float, b: float, eps: float
    ) -> Generator[Tuple[float, float, float, float], None, None]:
        """
        Yield (a, b, current_x_min, current_f_min) at each iteration.
        """
        pass


class DichotomyOptimizer(IterativeOptimizer):
    def __init__(self, max_iterations: int = 1000) -> None:
        super().__init__("Dichotomy", max_iterations)

    def _generate_steps(
        self, func: ObjectiveFunction, a: float, b: float, eps: float
    ) -> Generator[Tuple[float, float, float, float], None, None]:

        delta = eps / 10.0

        while True:
            mid = (a + b) / 2.0
            x1, x2 = mid - delta, mid + delta
            f1, f2 = func(x1), func(x2)

            if f1 < f2:
                b = x2

```

```

        yield a, b, x1, f1
    else:
        a = x1
        yield a, b, x2, f2

class GoldenSectionOptimizer(IterativeOptimizer):
    def __init__(self, max_iterations: int = 1000) -> None:
        super().__init__("Golden Section", max_iterations)

    def _generate_steps(
        self, func: ObjectiveFunction, a: float, b: float, eps: float
    ) -> Generator[Tuple[float, float, float, float], None, None]:

        phi = (np.sqrt(5) - 1) / 2.0
        c = b - phi * (b - a)
        d = a + phi * (b - a)
        fc = func(c)
        fd = func(d)

        while True:
            if fc < fd:
                b, d, fd = d, c, fc
                c = b - phi * (b - a)
                fc = func(c)
            else:
                a, c, fc = c, d, fd
                d = a + phi * (b - a)
                fd = func(d)

            if fc < fd:
                yield a, b, c, fc
            else:
                yield a, b, d, fd

class FibonacciOptimizer(IterativeOptimizer):
    """Fibonacci search."""

    def __init__(self, max_iterations: int = 1000) -> None:
        super().__init__("Fibonacci", max_iterations)

```

```

@staticmethod
def _fib_sequence_up_to(n: int) -> list[int]:
    """Generate Fibonacci numbers up to at least n."""
    fibs = [1, 1]
    while fibs[-1] < n:
        fibs.append(fibs[-1] + fibs[-2])
    return fibs

def _generate_steps(
    self, func: ObjectiveFunction, a: float, b: float, eps: float
) -> Generator[Tuple[float, float, float, float], None, None]:

    n_estimate = int(np.ceil((b - a) / eps))
    fibs = self._fib_sequence_up_to(n_estimate)
    n = len(fibs) - 1
    n = min(n, self.max_iterations)

    assert n >= 2, "Fibonacci: insufficient sequence length."

    c = a + (fibs[n - 2] / fibs[n]) * (b - a)
    d = a + (fibs[n - 1] / fibs[n]) * (b - a)
    fc = func(c)
    fd = func(d)

    for k in range(1, n):
        last_step = (k == n - 1)

        if fc < fd:
            b, d, fd = d, c, fc
            if last_step:
                c = (a + b) / 2.0 - eps / 10.0
            else:
                c = a + (fibs[n - k - 2] / fibs[n - k]) * (b - a)
            fc = func(c)
        else:
            a, c, fc = c, d, fd
            if last_step:
                d = (a + b) / 2.0 + eps / 10.0
            else:
                d = a + (fibs[n - k - 1] / fibs[n - k]) * (b - a)

```

```

        fd = func(d)

    if fc < fd:
        yield a, b, c, fc
    else:
        yield a, b, d, fd

class ParabolaOptimizer(IterativeOptimizer):
    """Parabolic interpolation search."""

    def __init__(self, max_iterations: int = 500) -> None:
        super().__init__("Parabola", max_iterations)

    def _generate_steps(
        self, func: ObjectiveFunction, a: float, b: float, eps: float
    ) -> Generator[Tuple[float, float, float, float], None, None]:

        x0, x2 = a, b
        x1 = (a + b) / 2.0
        f0, f1, f2 = func(x0), func(x1), func(x2)

        while True:
            numerator = (x1 - x0) ** 2 * (f1 - f2) - (x1 - x2) ** 2 * (f1 - f0)
            denominator = 2.0 * ((x1 - x0) * (f1 - f2) - (x1 - x2) * (f1 - f0))

            if abs(denominator) < 1e-12:
                yield x1 - 1e-15, x1 + 1e-15, x1, f1
                return

            x_new = x1 - numerator / denominator
            if not np.isfinite(x_new) or x_new <= x0 or x_new >= x2:
                yield x0, x2, x1, f1
                return

            f_new = func(x_new)

            if x_new < x1:
                if f_new < f1:
                    x2, f2 = x1, f1
                    x1, f1 = x_new, f_new

```

```

        else:
            x0, f0 = x_new, f_new
    else:
        if f_new < f1:
            x0, f0 = x1, f1
            x1, f1 = x_new, f_new
        else:
            x2, f2 = x_new, f_new

    yield x0, x2, x1, f1

# =====
# 4. Experiment manager
# =====

@contextmanager
def temporary_bounds(func: ObjectiveFunction, a: float, b: float):
    """Temporarily replace function bounds and restore them afterwards."""
    original = func.bounds
    func.bounds = (a, b)
    try:
        yield func
    finally:
        func.bounds = original

def run_multimodal_strategy(
    func: ObjectiveFunction,
    recon_eps: float,
    active_optimizer: Optimizer,
    fine_eps: float,
) -> pd.DataFrame:
    """
    Two-stage strategy for the multimodal function from the laboratory task.

    1. Passive search with coarse precision to obtain  $[x_{\text{recon}} \pm 2h]$ 
    2. Active method on the narrowed interval
    3. The same active method on the full interval

    Returns a comparison DataFrame.

```

```

"""
recon = PassiveSearchOptimizer(max_evaluations=10_000_000)
rows = []

func.reset_count()
recon_result = recon.minimize(func, recon_eps)
recon_calls = func.call_count

a_orig, b_orig = func.get_bounds()
h = (b_orig - a_orig) / recon_result.n_iterations
narrow_a = max(a_orig, recon_result.x_min - 2 * h)
narrow_b = min(b_orig, recon_result.x_min + 2 * h)

with temporary_bounds(func, narrow_a, narrow_b):
    func.reset_count()
    narrow_result = active_optimizer.minimize(func, fine_eps)

rows.append({
    "strategy": f"recon(eps={recon_eps:.0e}) + {active_optimizer.name}(eps={fine_ep
    "recon_calls": recon_calls,
    "active_calls": narrow_result.n_calls,
    "total_calls": recon_calls + narrow_result.n_calls,
    "x_min": narrow_result.x_min,
    "f_min": narrow_result.f_min,
    "converged": narrow_result.converged,
    "note": f"interval narrowed to [{narrow_a:.4f}, {narrow_b:.4f}]",
})

func.reset_count()
blind_result = active_optimizer.minimize(func, fine_eps)

rows.append({
    "strategy": f"{active_optimizer.name}(eps={fine_eps:.0e}), no recon",
    "recon_calls": 0,
    "active_calls": blind_result.n_calls,
    "total_calls": blind_result.n_calls,
    "x_min": blind_result.x_min,
    "f_min": blind_result.f_min,
    "converged": blind_result.converged,
    "note": f"full interval [{a_orig:.4f}, {b_orig:.4f}]",
})

```

```

return pd.DataFrame(rows)

class OptimizationExperiment:
    """
    Runs all function/method/epsilon combinations and stores structured results.
    """

    def __init__(
        self,
        optimizers: List[Optimizer],
        functions: List[ObjectiveFunction],
        output_dir: str | Path = "outputs",
    ) -> None:
        self.optimizers = optimizers
        self.functions = functions
        self.output_dir = Path(output_dir)
        self.plots_dir = self.output_dir / "plots"
        self.tables_dir = self.output_dir / "tables"
        self.output_dir.mkdir(parents=True, exist_ok=True)
        self.plots_dir.mkdir(parents=True, exist_ok=True)
        self.tables_dir.mkdir(parents=True, exist_ok=True)
        self.results_data: List[dict] = []

    def run(self, epsilon_list: List[float]) -> None:
        """Run all experiment combinations and store successes and failures."""
        self.results_data = []

        for func in self.functions:
            for opt in self.optimizers:
                for eps in epsilon_list:
                    func.reset_count()

                    try:
                        result = opt.minimize(func, eps)
                        self.results_data.append(
                            {
                                "function": func.name,
                                "is_unimodal": func.is_unimodal,
                                "optimizer": opt.name,

```



```

        "epsilon": float(eps),
        "iterations": result.n_iterations,
        "calls": result.n_calls,
        "x_min": result.x_min,
        "f_min": result.f_min,
        "history": result.history,
        "history_iterations": result.history_iterations,
        "history_available": result.history_available,
        "converged": result.converged,
        "status": "success" if result.converged else "warning",
        "message": result.message,
    }
)
except NotImplementedError as error:
    self.results_data.append(
        {
            "function": func.name,
            "is_unimodal": func.is_unimodal,
            "optimizer": opt.name,
            "epsilon": float(eps),
            "iterations": np.nan,
            "calls": np.nan,
            "x_min": np.nan,
            "f_min": np.nan,
            "history": [],
            "history_iterations": [],
            "history_available": False,
            "converged": False,
            "status": "todo",
            "message": str(error),
        }
    )
except Exception as error:
    self.results_data.append(
        {
            "function": func.name,
            "is_unimodal": func.is_unimodal,
            "optimizer": opt.name,
            "epsilon": float(eps),
            "iterations": np.nan,
            "calls": np.nan,

```

```

        "x_min": np.nan,
        "f_min": np.nan,
        "history": [],
        "history_iterations": [],
        "history_available": False,
        "converged": False,
        "status": "error",
        "message": str(error),
    }
)

def results_dataframe(self) -> pd.DataFrame:
    """Return all recorded runs as a DataFrame."""
    if not self.results_data:
        return pd.DataFrame()
    return pd.DataFrame(self.results_data)

def build_summary_table(self, include_failed: bool = True) -> pd.DataFrame:
    """Build a clean, numbered table for console output and export."""
    result_df = self.results_dataframe()
    if result_df.empty:
        return pd.DataFrame()

    if not include_failed:
        result_df = result_df[result_df["status"] == "success"].copy()

    result_df = result_df.sort_values(by=["function", "optimizer", "epsilon"], ascending=True)
    table = pd.DataFrame(
        {
            "No.": np.arange(1, len(result_df) + 1),
            "Function": result_df["function"],
            "Optimizer": result_df["optimizer"],
            "Epsilon": result_df["epsilon"],
            "Iterations": result_df["iterations"],
            "Function calls": result_df["calls"],
            "x_min": result_df["x_min"],
            "f(x_min)": result_df["f_min"],
            "Status": result_df["status"],
            "Comment": result_df["message"],
        }
    )

```

```

    )
    return table

def print_summary_table(self, include_failed: bool = True) -> None:
    """Print the summary table with numbered rows and clear column names."""
    table = self.build_summary_table(include_failed=include_failed)
    if table.empty:
        print("No experiment data available.")
        return
    print(table.to_string(index=False))

def save_summary_table(self, filename: str = "experiment_summary.csv", include_fail
    """Save the summary table to CSV for future report integration."""
    table = self.build_summary_table(include_failed=include_failed)
    output_path = self.tables_dir / filename
    table.to_csv(output_path, index=False)
    return output_path

def save_tables_by_function(self, only_unimodal: bool = True) -> List[Path]:
    """
    Save separate numbered tables for each function.

    This is convenient for the report, where tables are usually discussed
    function by function.
    """
    result_df = self.results_dataframe()
    if result_df.empty:
        return []

    result_df = result_df[result_df["status"] == "success"].copy()
    if only_unimodal:
        result_df = result_df[result_df["is_unimodal"]].copy()

    saved_paths: List[Path] = []
    for function_name in result_df["function"].unique():
        df_func = result_df[result_df["function"] == function_name].copy()
        df_func = df_func.sort_values(by=["optimizer", "epsilon"], ascending=[True,
        table = pd.DataFrame(
            {
                "No.": np.arange(1, len(df_func) + 1),
                "Optimizer": df_func["optimizer"],

```

```

        "Epsilon": df_func["epsilon"],
        "Iterations": df_func["iterations"],
        "Function calls": df_func["calls"],
        "x_min": df_func["x_min"],
        "f(x_min)": df_func["f_min"],
        "Status": df_func["status"],
        "Comment": df_func["message"],
    }
)
output_path = self.tables_dir / f"{self._slugify(function_name)}_table.csv"
table.to_csv(output_path, index=False)
saved_paths.append(output_path)

return saved_paths

def save_report_tables(self, only_unimodal: bool = True) -> List[Path]:
    """
    Save compact LaTeX tables intended for direct inclusion into Report.tex.

    The exported tables focus on the quantities required by the laboratory
    task: epsilon, iteration count and function-call count.
    """
    result_df = self.results_dataframe()
    if result_df.empty:
        return []

    if only_unimodal:
        result_df = result_df[result_df["is_unimodal"]].copy()

    saved_paths: List[Path] = []
    for function_name in result_df["function"].unique():
        df_func = result_df[result_df["function"] == function_name].copy()
        df_func = df_func.sort_values(by=["optimizer", "epsilon"], ascending=[True,
        table = pd.DataFrame(
            {
                "No.": np.arange(1, len(df_func) + 1),
                "Method": df_func["optimizer"],
                "Epsilon": df_func["epsilon"],
                "Iterations": df_func["iterations"],
                "Function calls": df_func["calls"],
                "Status": df_func["status"],

```

```

        }
    )

    latex = self._dataframe_to_longtable(table)

    output_path = self.tables_dir / f"{self._slugify(function_name)}_table.tex"
    output_path.write_text(latex, encoding="utf-8")
    saved_paths.append(output_path)

    return saved_paths

    @staticmethod
    def _format_table_value(value: object) -> str:
        if isinstance(value, (float, np.floating)):
            if value == 0:
                return "0"
            if abs(value) < 1e-2 or abs(value) >= 1e3:
                return f"{value:.0e}"
            return f"{value:g}"
        text = str(value)
        return (
            text.replace("\\", "\\textbackslash{")
            .replace("&", "\\&")
            .replace("%", "\\%")
            .replace("_", "\\_")
            .replace("#", "\\#")
            .replace("{", "\\{")
            .replace("}", "\\}")
        )

    @classmethod
    def _dataframe_to_longtable(cls, table: pd.DataFrame) -> str:
        columns = list(table.columns)
        column_spec = "rllrrl"

        header = " & ".join(cls._format_table_value(column) for column in columns) + r"
        body_lines = []
        for row in table.itertuples(index=False, name=None):
            formatted_row = " & ".join(cls._format_table_value(value) for value in row)
            body_lines.append(formatted_row + r" \\"

```

```

lines = [
    r"\begin{longtable}{\" + column_spec + \"}\",
    r"\toprule\",
    header,
    r"\midrule\",
    r"\endfirsthead\",
    r"\toprule\",
    header,
    r"\midrule\",
    r"\endhead\",
    r"\midrule\",
    r"\multicolumn{6}{r}{\textit{Continued on the next page}} \\\",
    r"\midrule\",
    r"\endfoot\",
    r"\bottomrule\",
    r"\endlastfoot\",
    *body_lines,
    r"\end{longtable}\",
]
return "\n".join(lines) + "\n"

def plot_metrics(self, only_unimodal: bool = True, show: bool = False) -> List[Path]
"""
Plot epsilon vs iterations and epsilon vs function calls for each function.
"""
result_df = self.results_dataframe()
if result_df.empty:
    return []

result_df = result_df[result_df["status"] == "success"].copy()
if only_unimodal:
    result_df = result_df[result_df["is_unimodal"]].copy()

saved_paths: List[Path] = []
for function_name in result_df["function"].unique():
    df_func = result_df[result_df["function"] == function_name].sort_values("epsilon")
    if df_func.empty:
        continue

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    for optimizer_name in df_func["optimizer"].unique():

```

```

df_opt = df_func[df_func["optimizer"] == optimizer_name].sort_values("e
axes[0].plot(
    df_opt["epsilon"],
    df_opt["iterations"],
    marker="o",
    linewidth=2,
    label=optimizer_name,
)
axes[1].plot(
    df_opt["epsilon"],
    df_opt["calls"],
    marker="o",
    linewidth=2,
    label=optimizer_name,
)

axes[0].set_xscale("log")
axes[1].set_xscale("log")
axes[0].set_yscale("log")
axes[1].set_yscale("log")
axes[0].set_xlabel("Epsilon")
axes[1].set_xlabel("Epsilon")
axes[0].set_ylabel("Iterations")
axes[1].set_ylabel("Function evaluations")
axes[0].set_title(f"Iterations vs epsilon: {function_name}")
axes[1].set_title(f"Function evaluations vs epsilon: {function_name}")

for ax in axes:
    ax.grid(True, which="both", linestyle="--", alpha=0.5)
    ax.legend()
    ax.spines["left"].set_visible(True)
    ax.spines["bottom"].set_visible(True)
    ax.spines["left"].set_color("black")
    ax.spines["bottom"].set_color("black")

fig.suptitle(f"Efficiency comparison for {function_name}", fontsize=13)
fig.tight_layout()

output_path = self.plots_dir / f"{self._slugify(function_name)}_metrics.png"
fig.savefig(output_path, dpi=200, bbox_inches="tight")
saved_paths.append(output_path)

```

```

        if show:
            plt.show()
        plt.close(fig)

    return saved_paths

def plot_interval_dynamics(
    self,
    function_name: str,
    optimizer_name: str,
    eps: float,
    show: bool = False,
) -> Path:
    """
    Plot the lower and upper bounds of the uncertainty interval by iteration.

    If the selected method does not expose interval history yet, the function
    still generates an informational placeholder plot.
    """
    result_df = self.results_dataframe()
    if result_df.empty:
        raise ValueError("No experiment data available.")

    mask = (
        (result_df["function"] == function_name)
        & (result_df["optimizer"] == optimizer_name)
        & np.isclose(result_df["epsilon"], eps)
    )
    row = result_df[mask]
    if row.empty:
        raise ValueError("Requested experiment combination was not found.")

    row = row.iloc[0]
    fig, ax = plt.subplots(figsize=(10, 6))

    if bool(row["history_available"]) and len(row["history"]) >= 2:
        history = row["history"]
        stored_iterations = row.get("history_iterations", [])
        if stored_iterations and len(stored_iterations) == len(history):
            iterations = np.asarray(stored_iterations)

```



```

        else:
            iterations = np.arange(len(history))
            lower_bounds = [interval[0] for interval in history]
            upper_bounds = [interval[1] for interval in history]

            ax.plot(iterations, lower_bounds, marker="o", label="Lower bound a")
            ax.plot(iterations, upper_bounds, marker="o", label="Upper bound b")
            ax.fill_between(iterations, lower_bounds, upper_bounds, alpha=0.25, label="")
        else:
            ax.text(
                0.5,
                0.5,
                "Interval history is not available for this method.",
                transform=ax.transAxes,
                ha="center",
                va="center",
                fontsize=11,
            )

        ax.set_xlabel("Iteration")
        ax.set_ylabel("Interval boundary value")
        ax.set_title(f"Interval dynamics: {optimizer_name}, {function_name}, eps={eps:g}")
        ax.grid(True, linestyle="--", alpha=0.5)
        ax.axhline(0.0, color="black", linewidth=0.8)
        labeled = [h for h in ax.get_legend_handles_labels()[0]]
        if labeled:
            ax.legend(loc="best")

        output_path = self.plots_dir / (
            f"{self._slugify(function_name)}_{self._slugify(optimizer_name)}_eps_{self.eps:g}"
        )
        fig.savefig(output_path, dpi=200, bbox_inches="tight")

        if show:
            plt.show()
        plt.close(fig)
        return output_path

    @staticmethod
    def _slugify(value: str) -> str:
        cleaned = "".join(ch.lower() if ch.isalnum() else "_" for ch in value)

```

```

        while "__" in cleaned:
            cleaned = cleaned.replace("__", "_")
        return cleaned.strip("_")

# =====
# 5. Demo entry point
# =====

def build_demo_experiment() -> OptimizationExperiment:
    """Create the full experiment configuration used in the laboratory work."""
    functions: List[ObjectiveFunction] = [
        GoodUnimodalFunction(),
        PlateauAsymmetricUnimodalFunction(),
        SecondSpecialUnimodalFunction(),
        MultimodalFunction()
    ]

    optimizers: List[Optimizer] = [
        PassiveSearchOptimizer(),
        BrentOptimizer(),
        DichotomyOptimizer(),
        GoldenSectionOptimizer(),
        FibonacciOptimizer(),
        ParabolaOptimizer()
    ]

    return OptimizationExperiment(optimizers=optimizers, functions=functions)

def main() -> None:
    """Run the full experiment suite and save all generated artifacts."""
    experiment = build_demo_experiment()

    experiment.run(DEFAULT_EPSILONS)
    experiment.print_summary_table(include_failed=True)

    summary_path = experiment.save_summary_table()
    function_table_paths = experiment.save_tables_by_function(only_unimodal=True)
    report_table_paths = experiment.save_report_tables(only_unimodal=True)

```

```

plot_paths = experiment.plot_metrics(only_unimodal=True, show=False)

df = experiment.results_dataframe()
for _, record in df[df["status"] == "success"].iterrows():
    experiment.plot_interval_dynamics(
        function_name=record["function"],
        optimizer_name=record["optimizer"],
        eps=float(record["epsilon"]),
        show=False,
    )

multimodal_func = MultimodalFunction()
active_opt = GoldenSectionOptimizer()

print("\n=== Multimodal strategy: recon + active vs blind ===")
strategy_df = run_multimodal_strategy(
    func=multimodal_func,
    recon_eps=1e-1,
    active_optimizer=active_opt,
    fine_eps=1e-6,
)
print(strategy_df.to_string(index=False))
strategy_path = experiment.tables_dir / "multimodal_strategy_comparison.csv"
strategy_df.to_csv(strategy_path, index=False)

print(f"\nSummary table saved to: {summary_path}")
print("Function tables:")
for path in function_table_paths:
    print(f" - {path}")
print("Report tables:")
for path in report_table_paths:
    print(f" - {path}")
print("Metric plots:")
for path in plot_paths:
    print(f" - {path}")
print(f"Multimodal strategy table saved to: {strategy_path}")

if __name__ == "__main__":
    main()

```